

PROJECT PROGRAMMEREN 2020

GAME SHAPE LAUNCHER IN C++

prof. dr. ir. Filip De Turck

pgm@lists.ugent.be

2de Bachelor Ingenieurswetenschappen:
Computerwetenschappen en Elektrotechniek

Academiejaar 2019-2020

INHOUDSOPGAVE

1	Introductie	1
1.1	Doel van het project	1
1.2	Game programming	1
1.2.1	Game loop	1
1.2.2	Bewegende beelden	2
1.2.3	Collision detection	2
1.3	Entity-component-system framework	2
2	Opgave	3
2.1	Het spel	3
2.2	Menu	3
3	Implementatie	6
3.1	Het coördinatenstelsel	6
3.2	Entity-component-system framework	6
3.2.1	Component	6
3.2.2	Entity	6
3.2.3	System	7
3.2.4	Engine	7
3.2.5	EntityStream	8
3.3	Voorstelling level	8
3.4	Voorstelling highscore file	8
3.5	Componenten	9
3.6	Systems	9
3.7	Hulpklassen	11
3.7.1	Point	11
3.7.2	Context	11
3.7.3	Allkit	11
4	Bestanden	13
4.1	Headerbestanden	13
4.2	Implementatiebestanden	13
4.3	Assets	14
5	Compileren	15
6	Stappenplan	16
7	Tips en opmerkingen	17
8	Indienen	18
9	Appendix: memory leaks C++	19

1 INTRODUCTIE

Hieronder wordt kort het doel van het project geschetst en wordt een inleiding gegeven over een aantal aspecten die typisch aan bod komen bij game programming. Het advies is om dit zeker door te nemen zodat er voldoende achtergrondkennis aanwezig is om met de opgave te starten.

1.1 Doel van het project

In dit project is het de bedoeling om een spel genaamd 'Shape Launcher' te implementeren in C++. Dit spel is een eenvoudige variant van het wereldberoemde Angry Birds¹. De uitwerking van het project dient te gebeuren in groepen van twee personen. Het doel is om zoveel mogelijk aspecten van softwareontwikkeling aan bod te laten komen. Hierbij denken we behalve het programmeren zelf aan het gebruik van de Visual Studio Code editor, de integratie van bestaande libraries en het efficiënt samenwerken aan een project door middel van Git. Het project is gequoteerd op 5 van de 20 punten voor het vak Programmeren.

1.2 Game programming

Het programmeren van games verschilt op een aantal vlakken van het programmeren van traditionele applicaties. Hieronder worden de belangrijkste zaken overlopen.

1.2.1 Game loop

De game loop is het belangrijkste onderdeel van het spel aangezien deze ervoor zorgt dat het spel vlot blijft lopen, onafhankelijk van het feit of de gebruiker al dan niet input genereert. Dit in tegenstelling tot meer traditionele softwareprogramma's zoals tekstverwerkers of browsers die enkel reageren wanneer de gebruiker een actie triggert, bijvoorbeeld het tekenen van een menu wanneer de gebruiker op een knop klikt. Een game loop in pseudocode zou er als volgt kunnen uitzien:

Algorithm 1 Een generiek voorbeeld van een game loop.

- 1: **while** user does not exit **do**
 - 2: Capture user input
 - 3: Run (simple) AI
 - 4: Move objects
 - 5: Resolve collisions
 - 6: Render graphics
-

Hierbij wordt achtereenvolgens gecontroleerd welke input er door de gebruiker gegeven is, wordt de logica van het spel en de artificiële intelligentie (AI) uitgevoerd, wordt de beweging van verschillende objecten uitgevoerd, worden eventuele botsingen afgehandeld en worden de objecten opnieuw getekend. De game loop kan uiteraard verder verfijnd worden naarmate de ontwikkeling van het spel verloopt, maar de meeste spellen zijn op dit basisprincipe gebaseerd.

Game loops kunnen anders geïmplementeerd worden afhankelijk van het platform waarvoor ze ontwikkeld zijn. Een spel voor game consoles kan bijvoorbeeld alle hardware resources naar eigen wens gebruiken, terwijl een spel voor Windows uitgevoerd moet worden binnen de beperkingen van de process scheduler. Verder is het zo dat de meeste spellen multi-threaded zijn, zodat de berekening van de AI losgekoppeld kan worden van de generatie van bewegende beelden in het spel. Dit creëert een kleine overhead, maar zorgt ervoor dat het spel efficiënter uitgevoerd kan worden op hyper-threaded of multicore processoren.

¹<https://www.angrybirds.com/>

1.2.2 Bewegende beelden

Het visuele aspect is bij games zeer belangrijk: er moet een vloeiend beeld weergegeven worden, wil men het spel er goed doen uitzien. De theorie rond hoeveel beelden per seconde het menselijk oog minimaal moet zien om iets als een vloeiende beweging waar te nemen, is veel ingewikkelder dan meestal wordt aangenomen. Er zijn een aantal vuistregels die in de meeste gevallen lijken te kloppen:

- Hoe groter het aantal frames per seconde (FPS), hoe vloeiender de beweging;
- Hoe kleiner het verschil tussen opeenvolgende frames, hoe vloeiender de beweging.

Het is gemakkelijk om de snelheid van de game loop ook in frames per seconde uit te drukken, zodat er een maat is om mee te vergelijken. Het aantal frames per seconde duidt dan eigenlijk aan hoeveel keer per seconde de toestand van het spel aangepast wordt. Het aantal gegenereerde FPS is enerzijds gelimiteerd door de hardware van de computer, maar anderzijds ook door hoe de berekeningen die in de game loop gebeuren. Wanneer een bepaalde actie bijvoorbeeld een groot aantal berekeningen vraagt, zal de iteratie van de game loop langer duren en zal het aantal zichtbare FPS (tijdelijk) dalen.

1.2.3 Collision detection

Bij games wordt dikwijls gesproken over “collision detection”, het detecteren van een botsing tussen twee of meerdere objecten. In elke iteratie van de game loop moet er, vóór het hertekenen van het nieuwe frame, gecontroleerd worden of er al dan niet een botsing veroorzaakt werd door het uitvoeren van de bewegingen van alle verschillende objecten. Dit kan bijvoorbeeld het tegen elkaar plaatsen zijn van de twee objecten in plaats van “in elkaar” of een botsing zijn die de objecten de tegenovergestelde richting opstuurt. Eens deze botsing gedetecteerd en opgelost is, kan het frame hertekend worden. In dit project zal collision detection zich beperken tot de tweedimensionale variant.

1.3 Entity-component-system framework

Uit ervaring hebben we geleerd dat een klassieke objectgeoriënteerde aanpak niet altijd de beste resultaten oplevert bij het ontwerpen van een game. Abstracties die het ene moment logisch lijken, kunnen later een hindernis vormen bij het toevoegen van nieuwe functionaliteiten. Voorbeelden hiervan zijn:

- Tekenen naar het scherm gebeurt door elke klasse een `Render()` methode te laten implementeren, maar dit zorgt ervoor dat dit aspect van het spel verspreid wordt over de volledige codebase, wat het moeilijker onderhoudbaar en uitbreidbaar maakt;
- Bij een objectgeoriënteerde aanpak wordt generieke functionaliteit geïmplementeerd in klassen waarvan de objecten kunnen overerven. Een entiteit in het spel zou bijvoorbeeld kunnen overerven van de klasse `InputController` om te kunnen reageren op input van de gebruiker. Deze werkwijze is echter problematisch wanneer bijvoorbeeld het gedrag van deze entiteit at runtime moet veranderen in functie van de AI en niet in functie van de gebruiker.

Bovenstaande problematiek wordt erkend in de spelindustrie en een van de voorgestelde oplossingen is het gebruik van een entity-component-system framework²³. Bij deze data-driven aanpak worden de verschillende spelobjecten niet meer voorgesteld als afzonderlijke klassen, maar als generieke entiteiten die samengesteld worden aan de hand van een bepaald aantal componenten. Deze componenten zijn data containers die specifieke data bevatten, zoals bijvoorbeeld de positie en snelheid van een entiteit. De functionaliteit die per entiteit uitgevoerd wordt, wordt vervolgens bepaald door de verschillende systemen die elk inwerken op een set van entiteiten met specifieke componenten. Voorbeelden hiervan zijn een systeem dat bewegingsvectoren toepast en een systeem dat entiteiten op het scherm kan tekenen.

²<http://www.richardlord.net/blog/why-use-an-entity-framework>

³<http://www.richardlord.net/blog/what-is-an-entity-framework>

2 OPGAVE

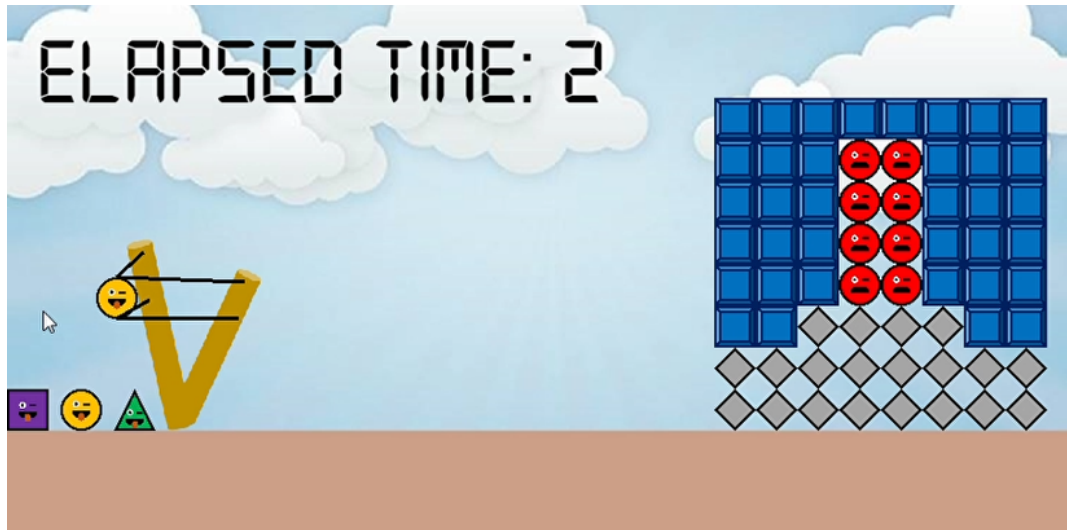
2.1 Het spel

In dit project zullen we het door ons bedachte spel 'Shape Launcher' programmeren. Een illustratie van het eindresultaat is weergegeven in Figuur 1. Een voorbeeldoplossing is ook beschikbaar gesteld zodat jullie het spel in beweging kunnen zien⁴. De gebruiker moet projectielen afvuren op een bepaalde constructie om zijn doel te bereiken, i.e. het zo snel mogelijk vernietigen van alle rode targets. Daarbij wordt de gebruiker verhinderd door blauwe boxen en grijze stenen. De blauwe boxen verdwijnen wanneer er een projectiel tegenaan vliegt. Hierdoor zakken alle boxen en/of stenen die boven de verwijderde box staan. De grijze stenen blijven altijd bestaan, om het spel enige moeilijkheid te geven. Het level weergegeven in de Figuur is zeer eenvoudig, juist omdat de stenen specifiek gepositioneerd staan. De besturing gebeurt aan de hand van de muis, i.e. het projectiel kan aangeklikt en verschoven worden, waarna het gelanceerd wordt. Onderweg kan het projectiel een specifieke actie uitvoeren. Dit gebeurt enkel wanneer de gebruiker de spatiebalk indrukt. Wanneer het spel afgelopen is, wordt er getoond of een nieuwe high score bereikt is, en kan de enter toets ingedrukt worden om terug te keren naar het menu. Elke uitvoering van het spel die leidt tot een nieuwe highscore, i.e. een zo laag mogelijke tijd, moet opgeslagen worden zodat deze opnieuw kan afgespeeld worden. Het begin van zo'n herhaling is ook te zien in de voorbeeldoplossing, waarbij volledig automatisch het spel opnieuw afgespeeld wordt. Let er tijdens de ontwikkeling van het spel op dat het steeds vlot gespeeld kan worden.

2.2 Menu

Rondom het spel moeten verschillende menus' voorzien worden. Figuur 2 toon welke menu's dit zijn. Er is een menu waar je een spel kan starten, een herhaling kan starten, of het spel kan afsluiten. Wanneer een spel gestart wordt, moet het mogelijk zijn om een level te kiezen dat ingeladen dient te worden. Wanneer het spel afgesloten wordt, keert men terug in dit level menu. Wanneer echter een herhaling dient afgespeeld te worden, moet het mogelijk zijn om een specifieke highscore te kiezen. Let erop dat de functionaliteit van een menu generiek is, i.e. je hebt een aantal knoppen waarachter een bepaalde functie schuilgaat. Denk er tijdens het implementeren over na hoe je de menu's zo generiek mogelijk kan implementeren. Indien je dit correct implementeert, is het toevoegen van een nieuw menu, bijvoorbeeld wanneer je het spel ooit zou willen uitbreiden, erg snel uit te voeren. **Daarnaast is het verplicht om een menu enkel te tekenen wanneer dit nodig is. Aangezien de gebruiker slechts om de zoveel tijd een knop indrukt en er dus niets in het scherm verandert, is het nutteloos om het scherm telkens leeg te maken en alles opnieuw te tekenen.**

⁴<https://cloud.ilabt.imec.be/index.php/s/9tRJKTsjnW8oGEX>



Figuur 1: Screenshots van 'Shape Launcher'.



Figuur 2: De verschillende menu's in het spel.

3 IMPLEMENTATIE

3.1 Het coördinatenstelsel

Aangezien 'Shape Launcher' een eenvoudig tweedimensionaal spel is, kan elk object op het scherm voorgesteld worden door een stel van (x, y) coördinaten. Het is daarom handig om een aantal conventies vast te leggen over het te kiezen assenstelsel. We zullen in dit project gebruikmaken van twee stelsels.

Het eerste stelsel komt overeen met het coördinatensysteem van Allegro (zie sectie 3.7.3) waarbij de x-coördinaat toeneemt naar rechts en de y-coördinaat toeneemt naar beneden. Bij het tekenen van afbeeldingen (bitmaps) door Allegro worden de (x, y) coördinaten van de linkerbovenhoek gebruikt. Wanneer Allegro dus gevraagd wordt om een bitmap in $(0,0)$ te tekenen, zal de linkerbovenhoek van de bitmap zich daar bevinden.



Figuur 3: Visualisatie van het assenstelsel.

Het klassieke assenstelsel, met een toename van de x- en y-coördinaat naar rechts en naar boven respectievelijk, moet in jullie code gebruikt worden om de entiteiten in het spel te positioneren. De linkeronderhoek van een object geeft dan zijn huidige positie weer. Dit wordt verduidelijkt in Figuur 3. Enkel wanneer je gaat tekenen, converteer je de posities naar de Allegro ruimte. Let op dat de ruimte waarin de entiteiten van het spel zich bevinden dus kleiner is dan de Allegro ruimte. Het is belangrijk (!) om deze conventies te volgen zodat de verbetering van jullie project vlot kan verlopen.

3.2 Entity-component-system framework

Meer informatie over een entity-component-system framework was te vinden in de eerder vermelde bronnen in Sectie 1.3. In dit project zullen we een basisversie van dit framework implementeren. Het framework bestaat uit een aantal belangrijke klassen, die hieronder kort overlopen worden.

3.2.1 Component

Een component is een heel eenvoudige data-container. De bedoeling is om conceptuele eigenschappen te bundelen in een klasse die overerft van de basisklasse Component. De pure virtuele functie `virtual Tag GetTag()= 0` dient geïmplementeerd te worden in elke subklasse om het type van een component te kunnen identificeren aan de hand van een enum waarde. Voor de eenvoud mogen de attributen van een component in dit project publiek gehouden worden, aangezien deze dikwijls zullen moeten uitgelezen worden.

3.2.2 Entity

Een Entity stelt om het even welk object voor in het spel, al dan niet zichtbaar. Entiteiten worden opgevuld door een verzameling Components toe te voegen die de eigenschappen van het object voorstellen. De belangrijkste methoden van de klasse Entity zijn:

- `void Add(Component* component):` Voegt een Component toe via een pointer naar deze component;

- `void Remove(Component* component)`: Verwijdert een Component via een pointer naar de te verwijderen component (verwijderen betekent niet vernietigen!);
- `Component* GetComponent(Component::Tag tag)`: Geeft een pointer naar een Component terug door opgeven van zijn Tag. Tag is een enum die gedefinieerd is in de Component klasse;
- `std::vector<Component*>& GetComponents()`: Geeft een referentie naar een vector van Component pointers terug die elk wijzen naar een component die hoort bij deze entiteit;
- `std::vector<Component::Tag>& GetTags()`: Geeft een referentie naar een vector met alle Tags (enum-waarden) van de componenten die horen bij deze entiteit.

3.2.3 System

Een System stelt een bepaald aspect van het spel voor, zoals beweging, collision detection en renderen naar het scherm. Alle systemen zullen overerven van de hoofdklasse System en zullen één of meerdere entiteiten verwerken in elke iteratie van de game loop. De klasse System beschikt over de volgende methoden:

- `virtual void Update() = 0`: Deze methode, die in elke iteratie van de game loop eenmaal wordt aangeroepen, bevat alle logica die het systeem dient uit te voeren. Deze methode moet daarom geïmplementeerd worden door alle overervende System klassen;
- `void SetEngine(Engine* engine)`: Dit is een eenvoudige setter waarmee de protected variabele engine ingesteld wordt om een pointer naar de engine beschikbaar te hebben in alle overervende systemen.

3.2.4 Engine

De Engine is de belangrijkste klasse van het framework, waarin alle klassen functioneel samenkomen. Zowel Entities als Systems worden toegevoegd aan de engine. De Engine heeft een Update()-methode die zal opgeroepen worden voor elke iteratie van de game loop, waarin elk systeem wordt aangeroepen. Systems zullen via de EntityStream (zie sectie 3.2.5) de nodige entiteiten kunnen opvragen om deze te verwerken. Let erop dat systemen nooit rechtstreeks met elkaar communiceren, aangezien deze volledig onafhankelijk van elkaar moeten kunnen functioneren. Communicatie kan indirect gebeuren via de context die wordt bijgehouden in een object van de Context klasse. De Engine beschikt over de volgende methoden:

- `void AddEntity(Entity* entity)`: Voegt een Entity toe aan de Engine via een pointer;
- `void UpdateEntity(Entity* entity, std::vector<Component::Tag>& tags, bool remove)`: Zorgt ervoor dat de EntityStream wordt aangepast overeenkomstig de tags die bij een entity zijn verwijderd of toegevoegd;
- `std::vector<Entity*>::iterator RemoveEntity(Entity* entity)`: Verwijdert een Entity van de Engine via een pointer en geeft een iterator naar de volgende entity terug;
- `void AddSystem(System* system)`: Voegt een System toe aan de Engine via een pointer;
- `std::vector<System*>::iterator RemoveSystem(System* system)`: Verwijdert een System van de Engine via een pointer en geeft een iterator naar het volgende system terug;
- `std::vector<Entity*>& GetEntities()`: Geeft een referentie naar een vector met pointers naar alle Entities uit de Engine terug;
- `EntityStream& GetEntityStream()`: Geeft een referentie terug naar het EntityStream object.
- `Context& GetContext()`: Geeft een referentie naar de Context klasse terug (zie sectie 3.7.2);
- `void Update()`: Roept om de beurt de Update()-methode aan op alle Systems die momenteel aan de Engine toegevoegd zijn;

Merk op dat er een cyclische afhankelijkheid bestaat tussen de klasse System en de klasse Engine, i.e. in de System klasse wordt een pointer naar de klasse Engine gebruikt en andersom. Dit kan opgelost worden door een forward declaration te gebruiken⁵.

3.2.5 EntityStream

Aan de EntityStream kan een set van Entities opgevraagd worden door op te geven over welke Componenten ze moeten beschikken. EntityStream moet dus op de hoogte zijn van alle Entities die toegevoegd en/of verwijderd worden aan/van de Engine. Dit is mogelijk door in de klasse Engine bij toevoegen of verwijderen van een Entity de onderstaande methoden van EntityStream op te roepen. Deze zullen de interne mapping in een `std::map` van (sleutel) Component::Tag naar (waarde) `std::set<Entity*>` aanpassen. EntityStream beschikt over de volgende methoden:

- `std::set<Entity*> WithTag(Component::Tag tag)`: Geeft een set terug met pointers naar de Entities die toegevoegd zijn aan de Engine én die een component met de opgegeven tag bevatten.
- `std::set<Entity*> WithTags(std::set<Component::Tag>& tags)`: Geeft een set terug met pointers naar de Entities die toegevoegd zijn aan de Engine en over alle opgegeven componenten beschikken. Dit betreft dus de doorsnede, niet de unie! Deze methode zal in dit project vermoedelijk niet opgeroepen worden, omdat ze dient om heel specifiek naar bepaalde entiteiten te zoeken. De bovenstaande `WithTag()` methode volstaat meestal aangezien dit een 'speelgoed' project is met weinig systemen en weinig verschillende entiteiten. Merk op dat deze methode er wel voor kan zorgen dat je geen downcast (door middel van `dynamic_cast<>`) moet uitvoeren wanneer je een component van een entity opvraagt, hetgeen wel nodig is wanneer je de `GetComponent()` functie gebruikt.
- `void EntityUpdated(Entity* entity, std::vector<Component::Tag>& tags, bool remove)`: Wordt aangeroepen door de Engine bij toevoegen / aanpassen / verwijderen van een Entity. Een pointer naar de Entity wordt aangeleverd via het argument. Daarmee wordt de interne mapping aangepast, zodat de `WithTag(s)`-methoden correct kunnen gebruikt worden.

3.3 Voorstelling level

Zoals eerder uitgelegd, bestaat een level uit rode targets, blauwe boxen en grijze stenen. De vorm van een level file is weergegeven in Figuur 4. Elk level bestaat uit een matrix met de karakters 'B' (box), 'S' (stone) en 'T' (target). De grootte van de matrix mag vrij gekozen worden, maar 8x8 lijkt geschikt voor de grootte van het scherm. Let er bij het uitlezen op dat er geen verkeerde karakters worden ingelezen of dat het een onvolledig level betreft.

```
B B B B B B B B
B B B T T B B B
B B B T T B B B
B B B T T B B B
B B B T T B B B
B B S S S S B B
S S S S S S S S
S S S S S S S S
```

Figuur 4: Voorstelling van een level file.

3.4 Voorstelling highscore file

De highscore files zijn nodig om een herhaling mogelijk te maken. Het patroon dat zij dienen te volgen staat weergegeven in Figuur 5. De file bestaat uit enkele tags die worden weergegeven met vierkante haken. Zo is er minimaal een LEVEL

⁵<https://en.cppreference.com/w/cpp/language/class>

tag, MISSILES tag en een SCORE tag. De ACTIONS tag wordt omschreven in de sectie over het MissileSystem hieronder. De LEVEL tag legt vast welk level gespeeld werd toen de highscore plaatsvond. De MISSILES tag legt op de eerste lijn vast welk type projectiel (geel rondje, paars vierkantje, groene driehoek) in de launcher geplaatst stond. In dit geval is dat type 1, dus het paarse vierkantje. De volgende lijnen leggen de bewegingen van de muis vast zoals waargenomen tijdens het voorbereiden van de lancering van het projectiel. De functies die je kan gebruiken om de locatie van de muis op te halen, staan beschreven in sectie 3.7.3. De END marker geeft aan wanneer het projectiel afgeschoten is. Vervolgens begint een volgende sequentie die de trace van het tweede projectiel vastlegt. Uiteindelijk is er de SCORE tag die vastlegt hoeveel seconden de gebruiker het spel gespeeld heeft. Let er ook hier op dat, wanneer een high score file ingelezen wordt, deze exacte representatie gevolgd wordt.

```
[LEVEL]
./levels/level1.txt
[MISSILES]
1
-1 -1
...
END
1
-1 0
...
END
[ACTIONS]
0
70 80
90 75
END
1
84 82
...
END
[SCORE]
60
```

Figuur 5: Voorstelling van een highscore file.

3.5 Componenten

De verschillende componenten die nodig zijn hangen uiteraard af van de functionaliteit in het spel en de manier waarop je zelf kiest om de aspecten in componenten op te delen. Er zijn allerlei logische opsplitsingen mogelijk, maar een volledige onlogische opsplitsing kan wel reflecteren in een iets lagere score. Let erop dat volgens het entity-component-system framework componenten enkel data containers zijn, i.e. het is niet mogelijk om gemeenschappelijke functionaliteit te abstraheren naar een hogere component. Om jullie inspiratie te geven, lijsten we hier op welke componenten we voor de voorbeeldoplossing gebruikt hebben: BoxComponent, CircleComponent, CurrentMissileComponent, MissileOneComponent (= het gele rondje), MissileTwoComponent (= het paarse vierkantje), MissileThreeComponent (= de groene driehoek), PolygonComponent, SpriteComponent, StoneComponent en TargetComponent.

3.6 Systems

Ook de verschillende systemen hangen wederom af van de functionaliteit in het spel. Houd er rekening mee dat de volgorde van het toevoegen van systemen aan de Engine van belang is; zo zal het tekenen van het frame telkens de laatste stap van de iteratie zijn. Wij hebben alvast de volgende systemen gebruikt voor de voorbeeldoplossing, maar je mag gerust andere systemen maken, zo lang deze nog een logische opsplitsing opleveren.

- **LauncherSystem:** Dit systeem is verantwoordelijk voor het beheer van de projectielen die in de wachtrij staan en

voor het projectiel dat gelanceerd gaat worden. Wanneer een nieuw spel gespeeld wordt, moet de wachtrij continu aangevuld worden met nieuwe projectielen. Het type projectiel mag willekeurig gekozen worden. Wanneer echter een herhaling wordt afgespeeld, mogen enkel die projectielen in de wachtrij staan die daadwerkelijk gebruikt zijn in de uitvoering van het spel. Dit systeem zorgt er ook voor dat de muisbewegingen toegepast worden op het huidige projectiel. Zoals eerder beschreven, worden deze muisbewegingen steeds vastgelegd, zodat bij een mogelijk nieuwe highscore, deze kunnen worden weggeschreven in een file. Om te weten of er op een projectiel geklikt is, moet je code voorzien die controleert of de locatie van de muis binnen de cirkel of binnen de veelhoek ligt. Voor de veelhoek kan je je baseren op de code die gebruikt maakt om een botsing te detecteren zoals deze in TargetSystem gebruikt wordt.

- **MissileSystem:** Dit systeem is verantwoordelijk voor de vlucht van het projectiel eens het afgeschoten is. Het zorgt ervoor dat de elastiek terugveert naar de normale stand en beweegt het projectiel verder aan de hand van de gekozen richting. Verder zal het, naargelang het type projectiel, een actie uitvoeren wanneer de gebruiker op de spatiebalk drukt. Welke actie uitgevoerd wordt per type projectiel mogen jullie zelf bepalen, maar dit kan bijvoorbeeld het volgende zijn: gele rondje versnelt in positieve y-richting waardoor grijze stenen ontweken kunnen worden, paarse vierkantje versnelt in negatieve y-richting, groene driehoekje is een soort van joker, i.e. het versnelt in x-richting zodat het op hoge snelheid tegen grijze stenen kan botsen, etc. Deze informatie moet ook gecapteerd worden in de highscore file door middel van de ACTIONS tag zoals weergegeven in Figuur 5. De eerste lijn geeft telkens de index van het projectiel weer, terwijl de lijnen daaropvolgend de coördinaten weergeven waarop een actie uitgevoerd is. Je mag zelf de verschillende acties kiezen, zolang elk type projectiel maar een specifieke actie heeft. We verwachten ook een korte uitleg hierover in het verslag.
- **TargetSystem:** Dit systeem is verantwoordelijk voor het berekenen van botsingen tussen het projectiel en de box en/of steen en/of rode target. Wanneer er een botsing plaatsvindt, wordt het geraakte element of de elementen oranje gekleurd en wordt het projectiel onmiddellijk verwijderd. Wanneer de impact van een binnenkomend projectiel een bepaalde threshold overschrijdt, verwijderen we ook opeenvolgende blokken op dezelfde rij (zijdelingse impact) of dezelfde kolom (impact van bovenaf). Hoeveel aaneengesloten blokken bij welke snelheid verwijderd worden, moet configureerbaar zijn in de Constants.h file. Hierdoor is het toch mogelijk om alle rode targets te verwijderen, ondanks dat de grijze stenen nooit verdwijnen. Extra toevoegingen mogen hier ook altijd geïmplementeerd worden, zoals bijvoorbeeld de situatie van een diagonale impact, maar dit is niet verplicht. In de opgavecode staat reeds volgende functie:

```
bool CheckCollision(std::vector<Point>& coordinates_poly_one, std::vector<Point>& coordinates_poly_two)
```

Je zal deze code desondanks wel moeten bestuderen om zodoende de functie in het LauncherSystem te implementeren die controleert of er op een willekeurige(!) veelhoek geklikt is. Dit gebruikt namelijk dezelfde wiskundige logica. Merk op dat we in dit project, voor de eenvoud, botsingen met ronde objecten constateren aan de hand van hun bounding box, i.e. het vierkant kader waarbinnen de cirkel gelegen is. Om een projectiel weg te schieten met de muis moet er wel rekening gehouden worden met de cirkelvorm. Om te controleren of je alles correct geïmplementeerd hebt, is het nodig om een DEBUG_MODE te voorzien. Deze constante is reeds in de Constants.h header geplaatst. Wanneer je deze constante op TRUE zet, moet je hetzelfde beeld krijgen als in Figuur 6. Merk ook hier de oranje gekleurde elementen op.

Let erop dat het detecteren van botsingen een vrij zware berekening is: wanneer je alle entiteiten in beschouwing gaat nemen, zal dit het spel enorm vertragen. Probeer daarom enkel de entiteiten te controleren die in aanmerking kunnen komen om te botsen, i.e. andere entiteiten die zeker buiten bereik liggen, kan je dan gewoon overslaan. Licht je keuzes toe in het verslag, zodat duidelijk is hoe je dit hebt aangepakt.

- **RenderSystem:** Zorgt voor het renderen van alle entiteiten die grafisch weergegeven dienen te worden en binnen het scherm liggen. **Dit systeem tekent ook de scores op het scherm.** Zoals het detecteren van botsingen, is ook renderen een zwaarder aspect van het spel. De Allegro library vereist dat we elk frame opnieuw construeren aangezien het onmogelijk is om enkel een specifiek object te verplaatsen. **Voor het spel kan dezelfde optimalisatie als voor de menu's doorgevoerd worden, i.e. er dient enkel getekend te worden wanneer er in het scherm iets veranderd is**, maar aangezien er daar vrijwel continu beweging is, is dit niet verplicht.



Figuur 6: Het spel in DEBUG modus.

3.7 Hulpklassen

3.7.1 Point

Dit is een hulpklasse die op twee manieren gebruikt wordt. Aan de ene kant is het een handige bundeling voor coördinaten of dimensies: het bestaat uit een publieke x- en y-component. Aan de andere kant wordt het gebruikt om wiskundige vectoren mee voor te stellen (een vector vanaf (0,0)). Om vectoriële bewerkingen mogelijk te maken in onder andere de collision berekeningen, moeten jullie nog een aantal functies implementeren in de Point klasse.

Point moet de volgende functies hebben:

- `double Length()`: Geef de lengte van de vector terug.
- `void Normalize()`: Normaliseer de vector, i.e. deel elke component door de lengte van de vector.
- Operator overloading
 - De operator `*` berekent de afstand tussen twee vectoren voorgesteld door Point objecten.
 - De operator `-` berekent het verschil tussen twee vectoren voorgesteld door Point objecten en geeft een nieuw Point object terug.
 - De operator `»` berekent de projectie van de ene vector op de andere vector (inwendig product).

3.7.2 Context

Dit is een hulpklasse waar een aantal variabelen in staan die aangepast en opgevraagd kunnen worden. Het bijhouden van de overkoepelende game state is daar slechts één voorbeeld van. Merk op dat, aangezien Context meegegeven wordt aan een Game en vervolgens aan een Engine, de variabele context via de engine opgevraagd kan worden in elk systeem. De inhoud van deze klasse mag je zelf bepalen, aangezien elk project een net iets andere approach zal nemen.

3.7.3 Allkit

Allkit is een wrapper rond de Allegro bibliotheek. Allegro laat toe om op relatief eenvoudige wijze naar het scherm te tekenen en een game te ontwikkelen. Via Allkit werd de library nog sterk vereenvoudigd en het is dan ook de bedoeling dat er gebruikt gemaakt wordt van deze wrapper waar mogelijk. Allegro-specifieke methoden, die typisch beginnen met "al_" moeten dus niet rechtstreeks gebruikt worden. De volgende methoden werden voorzien:

- `void Init()`: Initialiseert de Allegro bibliotheek. Dit moet aangeroepen worden alvorens ook maar iets te tekenen;

- `void Destroy()`: Ruimt alle interne structuren van Allegro op. Roep dit als laatste in je programma op, anders heb je sowieso memory leaks!;
- `void StartTimer()`: Deze methode start de achterliggende timer die 60 keer per seconde de game loop laat uitvoeren;
- `void StopTimer()`: Deze methode stopt de achterliggende timer;
- `void LoadLaunchSound()`: Een geluidsfragment inladen dat afgespeeld wordt bij het lanceren van een projectiel;
- `void PlayLaunchSound()`: Speelt het geluid af wanneer een projectiel gelanceerd wordt;
- `void NextEvent()`: Het wacht op het volgende event dat door Allegro gegenereerd wordt. Deze events bestaan uit timer events, toetsenbord events, muis events en display events;
- `bool IsArrowKeyUpPushed()`: Geeft aan of toets met pijltje naar boven ingedrukt is;
- `bool IsArrowKeyDownPushed()`: Geeft aan of toets met pijltje naar onder ingedrukt is;
- `bool IsEnterKeyPushed()`: Geeft aan of een van de enter toetsen ingedrukt is;
- `bool IsSpaceBarPushed()`: Geeft aan of de spatiebalk ingedrukt is;
- `bool IsMouseClicked()`: Geeft aan of er op de muis geklikt is;
- `bool IsMouseReleased()`: Geeft aan of de knop op de muis is losgelaten;
- `bool HasMouseMoved()`: Geeft aan of de muis bewogen heeft;
- `Point& GetMouse()`: Geeft de huidige locatie van de muis weer;
- `bool IsScreenActive()`: Geeft aan of het venster van het spel geselecteerd is;
- `bool IsTimerEvent()`: Geeft aan dat dit 1 van de 60 iteraties per seconde is. Deze methode wordt gebruikt om de engine met deze frequentie te doorlopen;
- `bool IsWindowClosed()`: Geeft true terug als het scherm werd gesloten door middel van het kruisje rechtsboven;
- `void ClearScreen()`: Geeft een zwart scherm weer.
- `void DrawOnScreen()`: Flit de bitmap in het geheugen, waar tot nog toe naar getekend werd, met die op het scherm. Elke tekenoperatie wordt namelijk eerst naar een bitmap in het geheugen gedaan. Deze techniek, beter gekend als buffering, zorgt ervoor dat er geen flikkering in het scherm komt bij het renderen;
- `void DrawLine(Point& a, Point& b)`: Tekent een lijn tussen punt A en punt B.
- `void DrawPoly(float dx, float dy, std::vector<Point>& body)`: Tekent lijnen tussen de punten vervat in het body argument;
- `void DrawString(std::string& str, Point& location, Color& color, Align align, bool huge_font)`: Tekent een string op een scherm met een bepaalde kleur en gealigneerd volgens de align enum. Het huge_font veld laat toe om een groot lettertype te gebruiken;
- `void DrawScaledBitmap(Sprite sprite, float sx, float sy, float sw, float sh, float dx, float dy, float dw, float dh)`: Tekent een herschaalde bitmap op positie (dx, dy);

Merk op dat Allkit als singleton werd geïmplementeerd. Dit kan je zien aan de copy assignment operator en de copy constructor die beiden 'delete' zijn. Opvragen van de instance gebeurt dus steeds met de statische methode `Allkit::Get()`, waarna je de nodige methoden kan oproepen.

4 BESTANDEN

Een deel van de code wordt bij de opgave meegegeven. Deze code is te vinden in de PGM repository⁶. Het is de bedoeling de meegeleverde bestanden waar nodig aan te vullen en uiteraard om de ontbrekende bestanden / klassen te implementeren. De onderstaande bestanden zijn geheel of gedeeltelijk gegeven. Als je niets hoeft aan te passen in een file mag je wel eventueel aanvullen als dat voor jouw oplossing nuttig is.

4.1 Headerbestanden

1. Allkit.h: volledig gegeven, hoeft je niets in aan te passen.
2. Color.h: volledig gegeven, hoeft je niets in aan te passen.
3. Component.h: declareer hier de basisklasse Component waarin alle mogelijke tags als enum waarde worden vastgelegd. Zet de kind-klassen best in aparte files.
4. Constants.h: Deze header file kan gebruikt worden om constanten in het spel vast te leggen. Het is uiteraard toegelaten om dit bestand verder aan te vullen.
5. Context.h: gebruik deze header om game state te bewaren.
6. Engine.h: publiek gedeelte gegeven, voeg eventuele private members toe.
7. Entity.h: publiek gedeelte gegeven, voeg eventuele private members toe.
8. EntityStream.h: publiek gedeelte gegeven, voeg eventuele private members toe.
9. Game.h: vul verder aan.
10. Menu.h: vul verder aan.
11. Point.h: de functies die hierboven vermeld zijn, dienen geïmplementeerd te worden.
12. Sprite.h: volledig gegeven, hoeft je niets in aan te passen.
13. System.h: declareer hier de basisklasse System.
14. TargetSystem.h: de signatuur van de methodes voor collision detectie zijn gegeven.

4.2 Implementatiebestanden

1. Allkit.cpp: reeds volledig geïmplementeerd, hoeft je niets in aan te passen.
2. Engine.cpp: de signatuur van de te implementeren methodes is reeds gegeven.
3. Entity.cpp: de signatuur van de te implementeren methodes is reeds gegeven.
4. EntityStream.cpp: de signatuur van de te implementeren methodes is reeds gegeven.
5. Game.cpp: vul verder aan.
6. Main.cpp: is volledig gegeven, maar mag aangepast worden.
7. Menu.cpp: een deel van de implementatie is reeds gegeven, vul de menu loop verder aan en zorg voor de aparte menu's op een generieke manier.
8. TargetSystem.cpp: het algoritme om botsingen tussen entiteiten te berekenen is volledig gegeven, vul dit system verder aan waar nodig.

⁶<https://github.ugent.be/Programmeren/Project>

4.3 Assets

1. De folders 'fonts', 'images' en 'sounds' bevatten de resources die gebruikt worden door de Allegro wrapper Allkit.
2. De folder 'levels' bevat de levels die we meegeven als aanzet. Je mag deze level files gerust aanpassen of zelf level files genereren.
3. De folder 'highscore' is momenteel leeg, maar dient dus de highscore files te bevatten zoals hierboven beschreven is.

5 COMPILEREN

De setup die jullie tijdens de labo's gebruikt hebben, zal ook voor het project gebruikt worden. De opgavecode bevat reeds alle correcte configuratie. Enkel de Allegro library moeten jullie nog toevoegen en dit kan op de volgende manier:

```
sudo add-apt-repository ppa:allegro/5.2
sudo apt-get update
sudo apt-get install liballegro5-dev
```

Voor Microsoft Windows gebruikers zijn er enkel additionele stappen nodig om het spel en de bijbehorende audio te kunnen weergeven. Deze stappen zijn samengevat in de algemene documentatie repository van het vak Programmeren⁷.

⁷<https://github.ugent.be/Programmeren/docs/blob/master/setup-project.md>

6 STAPPENPLAN

Hieronder worden enkele stappen opgesomd die een leidraad kunnen vormen om met het project te beginnen en het tot een goed einde te brengen.

1. Voer de installatiehandleiding zoals beschreven in de PGM repository uit. De meegeleverde code zou nu moeten compileren en uitvoeren. Indien dit zo is, wordt er bij het opstarten van het spel een zwart scherm weergegeven.
2. Implementeer de menu's zoals weergegeven in Figuur 2. Je kan deze stap ook zonder enig probleem als laatste doen, maar het maakt het uitvoeren van het spel eenvoudiger. Je kan immers snel een nieuw spel starten om nieuwe functionaliteit te testen zonder dat je het hele project opnieuw moet runnen.
3. Vervolledig eerst het entity-component-system framework zoals gedefinieerd is in sectie 3.2.
4. Implementeer alle componenten die nodig zijn, bijvoorbeeld dezelfde als gepresenteerd in sectie 3.5.
5. Implementeer vervolgens de logica van de systemen. Houd hierbij in het achterhoofd dat in elke iteratie volgende volgorde aangehouden wordt:

(a) LauncherSystem -> Update()

(b) MissileSystem -> Update()

(c) TargetSystem -> Update()

(d) RenderSystem -> Update()

In welke volgorde je de onderdelen van deze systemen implementeert, kan je zelf kiezen. Dit hoeft uiteraard niet sequentieel te zijn. Een goede start zou kunnen zijn om te analyseren wat er nodig is om het eerste scherm van het spel te renderen. Probeer zo stap per stap meer intelligentie in de verschillende systemen toe te voegen om uiteindelijk tot de volledige oplossing te komen.

7 TIPS EN OPMERKINGEN

Hieronder worden een aantal tips en opmerkingen gegeven waar best rekening mee gehouden wordt in dit project:

- Weet dat de basis implementatie zoals deze gevraagd wordt in deze opgave genoeg is om het maximum van de punten te halen. Uitbreidingen dienen dus slechts toegevoegd te worden eens alles werkt en enkel indien je dit zelf zou willen. Uitbreidingen zullen geen extra punten met zich meebrengen;
- Aangezien dit project met twee studenten gemaakt dient te worden, zal een vlotte samenwerking noodzakelijk zijn. Het gebruik van UGent(!) GitHub (<https://github.ugent.be/>) wordt hierbij sterk aangeraden, zolang de repository privaat(!) is. Het publiek plaatsen van de code zal gedetecteerd worden door onze interne monitoring tools en zal leiden tot puntenverlies;
- Begin niet onmiddellijk te programmeren. Lees eerst aandachtig de opgave en denk na hoe je de verschillende problemen zou oplossen;
- Het voordeel van een entity-component-system framework is dat je bepaalde systemen kan uitschakelen tijdens het debuggen, door ze tijdelijk niet aan de Engine toe te voegen;
- Maak zoveel mogelijk gebruik van de Standard Template Library (STL) (<http://www.cppreference.com/wiki/stl/start>);
- Alle opmerkingen bij de practica gelden ook hier: wees zuinig met geheugen, let op voor dangling pointers, geef alle gealloceerde geheugen ook weer vrij, controleer of bestanden wel correct geopend en gesloten worden, etc.;
- Als programmeerstijl werd de Google C++ Style Guide gebruikt (<https://google.github.io/styleguide/cppguide.html>). Vele andere opties waren echter mogelijk, één enkel 'juist' alternatief bestaat niet. We vragen wel om, in de kader van dit project, de gehanteerde stijl zoveel mogelijk te respecteren.
- Als referentiecompiler wordt de C++ compiler van de GNU Compiler Collection versie 8.3.0 gebruikt. Zorg er dus voor dat je project daarmee compileert en uitvoerbaar is. Het gebruik van C++11 / C++17 is toegestaan;
- Probeer compiler warnings te vermijden; deze duiden meestal op fouten die gemakkelijk op te lossen zijn;
- Vanzelfsprekend zijn er zaken waar deze opgave je vrij in laat; deze zijn naar eigen inzicht in te vullen. **Verduidelijk in elk geval je broncode met commentaar waar dit nuttig is;**
- Vragen over het project kunnen gesteld worden via het mailadres pgm@lists.ugent.be. Wij trachten alle vragen zo snel mogelijk te beantwoorden. Indien er een probleem is met de samenwerking tussen de groepsleden, gelieve dit dan zo snel mogelijk door te geven, zodat er een oplossing uitgewerkt kan worden.

8 INDIENEN

Het project wordt gemaakt in groepen van twee studenten. Om in te dienen zip je de broncodebestanden (.h en .cpp, eventueel samen met een mogelijke uitbreiding op het spel) samen met een verslag in een bestand en geef je dit bestand de naam Project_PGM_Groep_Naam1_Naam2.zip. Dit bestand wordt vervolgens ingediend via Indianio en dit ten laatste op zondag 17 mei 2020, 23u59. Het verslag is van het type PDF, is maximaal twee pagina's lang en omvat volgende zaken:

- De naam en richting van de groepsleden + Minerva groepsnummer
- Een korte uitleg over de belangrijkste ontwerpbeslissingen
- Bij het niet (correct) uitvoeren van het programma: waarom?
- De taakverdeling: wie heeft wat gedaan?

Als minimumvoorwaarde wordt vereist dat het project compileerbaar is met de GNU C++ compiler versie 8.3.0, zoniet wordt een nulscore toegekend. Wanneer het programma niet correct uitgevoerd kan worden, wordt er gevraagd hierover een korte toelichting te geven in het verslag.

9 APPENDIX: MEMORY LEAKS C++

Op Linux, Mac en in WSL is het heel eenvoudig om memory leaks en andere geheugenfouten zoals een “double free” op te sporen door middel van AddressSanitizer (ASan). Dit is een geheugen foutdetector origineel ontwikkeld door Google voor de Clang compiler die sinds versie 4.8 ook in GCC verwerkt zit.

ASan staat automatisch aan voor de meegegeven projectcode. Je kan die voor andere programma's zelf aanzetten door de flags “-fsanitize=address -fno-omit-frame-pointer” mee te geven met de compiler en de linker.

De compiler linkt alle geheugenoperaties zoals `malloc` en `free` door naar ASan, zodat deze alle geheugenoperaties kan bijhouden. Bij het afsluiten van het programma wordt er gekeken welk geheugen er aangevraagd is zonder het te verwijderen en rapporteert ASan alle memory leaks in het formaat dat hieronder te zien is.

```
=====
==22505==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 20000 byte(s) in 1 object(s) allocated from:
#0 0x7f9ad892a848 in __interceptor_malloc (/usr/lib/x86_64-linux-gnu/libasan.so.5+0xee848)
#1 0x55b9aede4800 in graphics_init /sysprog/game/graphics/
    opengl_game_renderer.c:364
#2 0x55b9aede4605 in graphics_alloc /sysprog/game/graphics/
    opengl_game_renderer.c:343
#3 0x55b9aedb75ee in main /sysprog/puzzle_bots_part2_main.cpp:43
#4 0x7f9ad6f32b96 in __libc_start_main (/lib/x86_64-linux-gnu/libc.so
    .6+0x21b96)
```

Deze stack trace toont aan welke code het geheugen aangemaakt heeft. Zoals je ziet worden alle geheugenoperaties uiteindelijk uitgevoerd door `libasan`, de geheugenchecker zelf. Je mag regel #0 dus overslaan. Het eigenlijke geheugenlek doet zich voor in de `graphics_init` functie in `opengl_game_renderer.c` op lijn 364. De rest van de stacktrace toont de volledige volgorde van oproepen: de `main` functie roept op lijn 43 de `graphics_alloc` functie op, die op lijn 343 de `graphics_init` functie oproept.

Bij het debuggen wordt een extern consolevenster gebruikt dat afsluit bij het stoppen van het programma. Hierdoor is het niet mogelijk om de memory leaks te zien na een debugsessie. Merk op dat de optie “start without debugging” de debugger toch opstart. Dit is een gekende fout in de C/C++ extensie van Visual Studio Code.